



Python Course 2025/26/2

Lecture 5: Error Handling, Files, Comprehensions

Course coordinator: Dr Mate Tejfel

Lecturer: Zoltan Kegli



Lecture Overview

Agenda

- Recap: functions and to-do app assumptions
- Error types and resilient mindset
- `try/except/else/finally` and `raise`
- File I/O: modes, context managers, common failures
- JSON persistence for structured data
- Comprehensions and generator expressions

Learning objective

Write Python programs that fail gracefully, keep data persistent, and stay concise without losing readability.

By the end, you can

Handle user and file errors, choose correct file modes, save state with `json`, and use comprehensions appropriately.



Recap

Functions and the to-do app baseline



Functions: Quick Refresher

Core terms

- Function: reusable block of logic
- Parameters: placeholders in definition
- Arguments: values at call time
- Return value: result sent back to caller
- Local scope: names created inside function

```
def calculate_area(length, width):  
    """Calculate rectangle area."""  
    area = length * width  
    return area  
  
result = calculate_area(5, 3)  
print(result) # 15
```

Functions improve reuse, testing, and readability.



Recap: To-Do App State

```
tasks = {
    1: {"description": "Learn Python basics", "status": "complete"},
    2: {"description": "Implement to-do app", "status": "pending"},
}

def add_task(description, status="pending"):
    task_id = len(tasks) + 1
    tasks[task_id] = {"description": description, "status": status}
    return task_id, tasks[task_id]
```

Assumptions we make

- User always enters valid IDs
- Keys always exist in dictionaries
- Program never restarts unexpectedly

Reality check: in-memory data disappears when the process ends. We need resilience and persistence.



Error Handling

From crashes to graceful behavior



What Could Go Wrong?

Invalid key access

```
tasks[99] # KeyError
```

Happens when users request non-existing task IDs.

Invalid numeric input

```
task_id = int(input("Enter Task ID: "))  
# "three" → ValueError
```

Common in CLI apps where input arrives as strings.

Engineering mindset: robust software starts with proactive pessimism: "What happens when assumptions fail?"

Resiliency



Errors

Problems that prevent intended behavior.

Exceptions

Runtime events that disrupt normal flow.

Resilient code

Continues safely or fails with clear user feedback.

Goal of this section: anticipate, catch, and handle exceptions without crashing the whole program.



Errors vs Exceptions vs Logic Bugs

```
# Syntax error (detected before execution)
# print("Hello"

# Runtime exception
print(10 / 0) # ZeroDivisionError

# Logic bug (runs, wrong result)
def add_numbers(a, b):
    return a - b
```

Quick distinction

- **SyntaxError** parser rejects invalid Python
- **Exception** code is valid but runtime context fails
- **Logic error** no crash, but wrong output

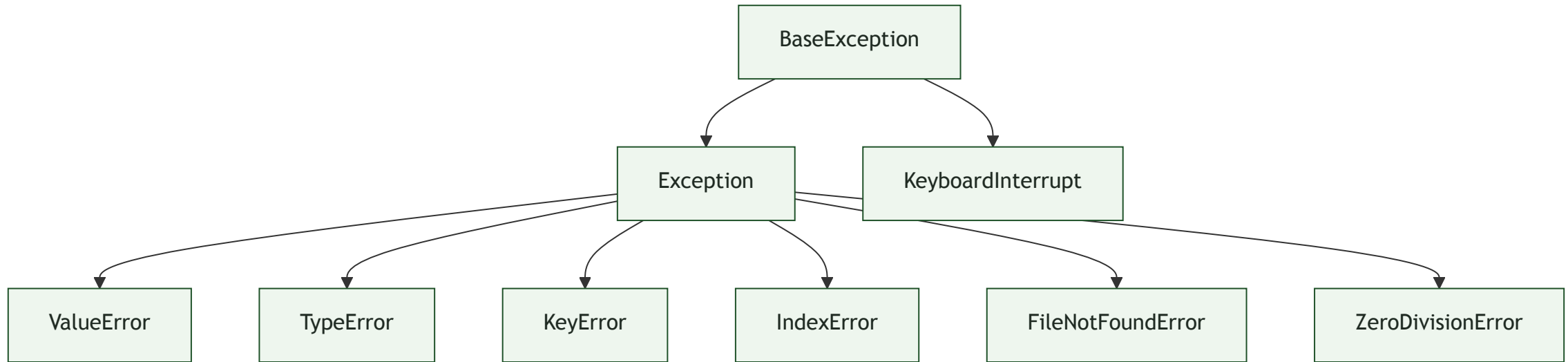


Exceptions We Already Met

Exception	Typical trigger	Example
<code>NameError</code>	Undefined variable in scope	<code>print(total_price)</code>
<code>TypeError</code>	Operation on incompatible types	<code>"3" + 5</code>
<code>ValueError</code>	Right type, wrong value	<code>int("abc")</code>
<code>IndexError</code> / <code>KeyError</code>	Missing position or key	<code>my_list[99]</code> , <code>my_dict["x"]</code>

Different exception types communicate different failure causes. Catching specifically makes fixes clearer.

Exception Hierarchy (Big Picture)



Catch the most specific exception you expect. Broad catches should be rare and intentional.



Handling Exceptions with try/except

```
try:
    x = int(input("Enter an integer: "))
    result = 10 / x
    print("10 /", x, "=", result)
except ValueError:
    print("Please enter a valid number!")
except ZeroDivisionError:
    print("Cannot divide by zero.")
```

Execution model

- Run risky code in `try`
- Jump to first matching `except`
- Skip other code in `try` after an exception
- If no exception: `except` blocks are skipped



else and finally

else

Runs only when the `try` block succeeds without any exception.

Good for post-processing successful results.

finally

Runs no matter what: success, handled failure, or re-raised exception.

Use for cleanup (files, sockets, temporary resources).

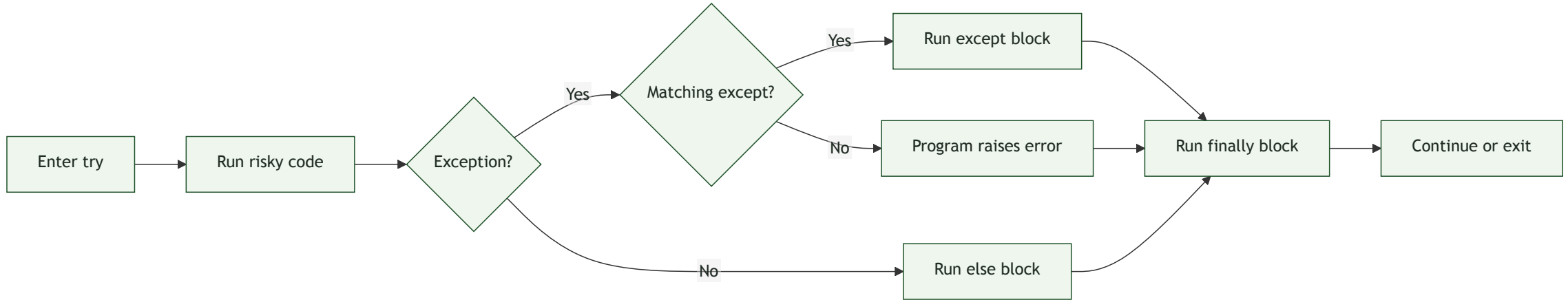
```
try:
    risky_operation()
except SomeError:
    handle_error()
else:
    process_success()
finally:
    cleanup_resources()
```

Example: safe_division()

```
def safe_division(numerator, denominator):  
    try:  
        num = float(numerator)  
        den = float(denominator)  
        result = num / den  
    except ValueError:  
        print("Invalid input: use numbers.")  
    except ZeroDivisionError:  
        print("Cannot divide by zero.")  
    else:  
        print(f"Success! Result: {result}")  
    finally:  
        print("Division attempt completed.")
```

Pattern: validate and execute in `try`, communicate failures in `except`, keep success-only output in `else`.

try/except/else/finally Flow



`finally` always runs, even when an exception is not handled locally.



raise: Signal Invalid States

```
def calculate_birth_year(age, current_year):  
    if age < 0:  
        raise ValueError("Age cannot be negative.")  
    return current_year - age  
  
try:  
    year = calculate_birth_year(-5, 2024)  
except ValueError as err:  
    print("Calculation failed:", err)
```

Why raise manually?

- Enforce function preconditions
- Fail early with clear messages
- Let calling code decide recovery strategy



Best Practices

Be specific

Catch known exceptions instead of broad `Exception`.

Handle meaningfully

Provide user-friendly messages and useful fallback behavior.

Do not overuse

Not every line needs a `try` block.

Common gotcha: bare `except:` catches almost everything, including `KeyboardInterrupt`.



Checkpoint: Error Handling

Q1

What is printed by this code: `try: int("abc") ... except ValueError: print("bad input")`?

- A) bad input
- B) ok
- C) no output
- D) The laptop enters meditation mode

Show Answer

Q2

What is risky about using bare `except:` ?

- A) It only catches ValueError
- B) It catches many exceptions, including interrupts you often want to propagate
- C) It works only on Tuesdays
- D) Python automatically ignores it

Show Answer



File I/O

Persistence and safe file access



Introduction to File I/O

Why files matter

- Persist data across program restarts
- Share data between tools
- Record logs and configuration

Common categories

- Text files: `.txt`, `.csv`, `.json`
- Binary files: `.jpg`, `.pdf`, `.exe`
- Core operations: open, read/write, close



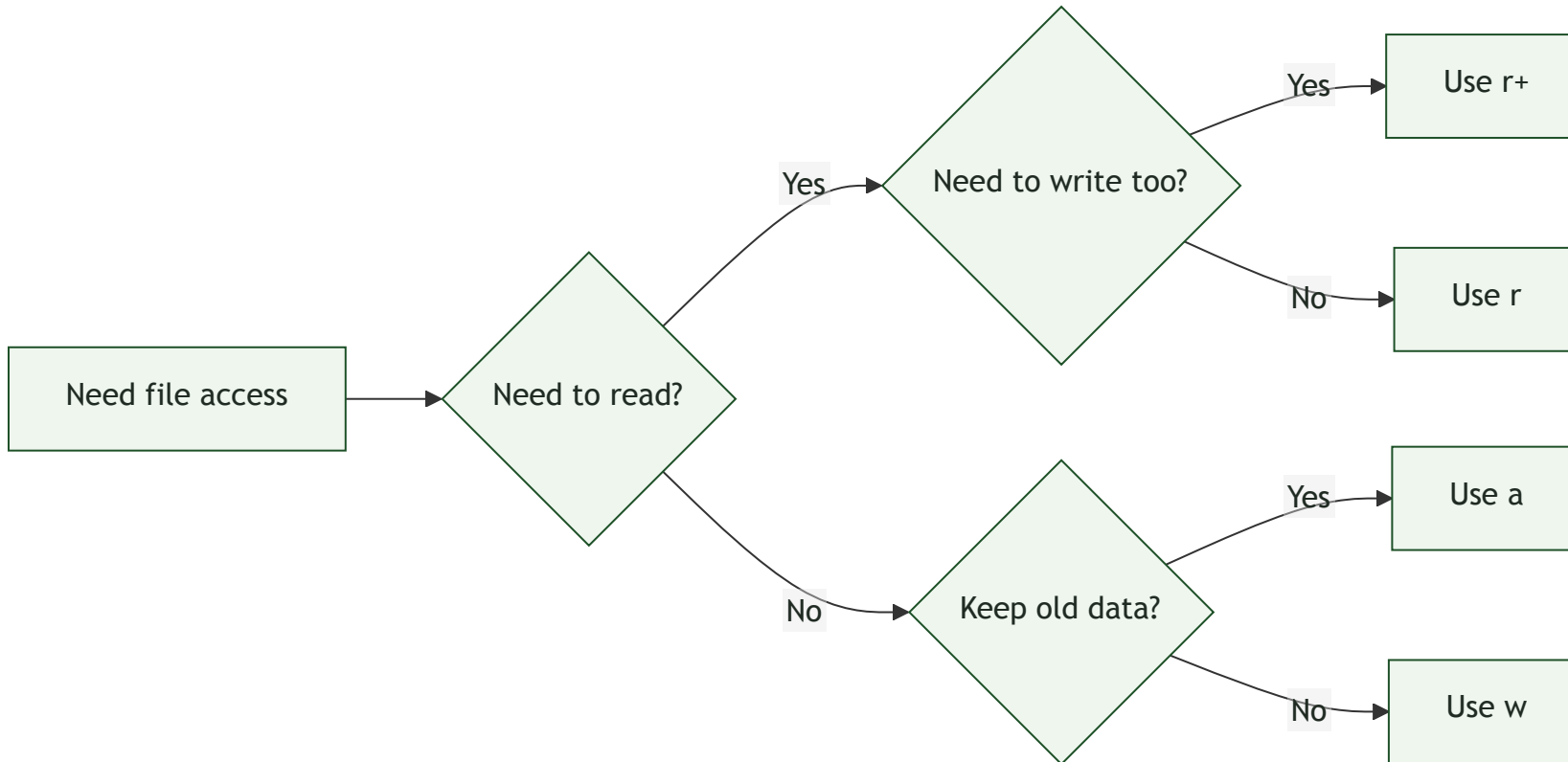
File Modes at a Glance

Mode	Purpose	If file exists	If file missing
r	Read only	Read from start	Raises FileNotFoundError
w	Write (truncate)	Overwrites all content	Creates new file
a	Append	Adds to end	Creates new file
r+	Read + write	Starts at beginning	Raises FileNotFoundError

Gotcha: `w` truncates immediately.



Which Mode Should You Use?



Start from intent: read, append, or replace. The mode should document your intent clearly.

Opening Files Safely with with

```
with open("data.txt", "r") as file:
    content = file.read()
    print(content)

with open("story.txt", "a") as file:
    file.write("New line\n")
```

Why context managers?

- Close files automatically
- Safer when exceptions occur
- Less boilerplate than manual `close()`

Fun fact: `with` uses `__enter__` and `__exit__` under the hood.

Reading and Writing Patterns



Reading

```
with open("story.txt", "r") as file:
    for line in file:
        print(line.strip())
```

Line-by-line loops scale better for large files.

Writing

```
lines = ["Line 1\n", "Line 2\n"]
with open("story.txt", "a") as file:
    file.writelines(lines)
```

Use explicit newlines when writing line-wise output.

Handling File Errors (EAFP)

```
try:
    with open("config.txt", "r") as cfg:
        settings = cfg.read()
except FileNotFoundError:
    print("Config not found, using defaults.")
```

Typical file exceptions

- `FileNotFoundError`
- `PermissionError`
- `IsADirectoryError`

EAFP: try the operation, then handle failure if it happens.

EAFP keeps code direct and avoids race conditions common in pre-check patterns.



Structured Persistence with JSON

Why JSON?

- Human-readable text format
- Maps directly to Python lists/dicts
- Ideal for app state and settings

Core API

`json.dump(data, file)`: Python object -> JSON text

`json.load(file)`: JSON text -> Python object



JSON Example

```
import json

student_data = {
    "name": "Alice",
    "id": 12345,
    "courses": ["History", "Math", "Physics"]
}

with open("data.json", "w") as f:
    json.dump(student_data, f, indent=4)

with open("data.json", "r") as f:
    loaded_data = json.load(f)
```

JSON avoids brittle manual parsing for nested structures.



Checkpoint: File I/O

Q3

What happens when you open an existing file with mode `"w"` ?

- A) Existing content is preserved
- B) Existing content is deleted, then writing starts
- C) Python asks for admin approval first
- D) It switches automatically to append mode

Show Answer

Q4

Why is `with open( ... )` preferred over manual `open()` / `close()` ?

- A) It can auto-close files even when exceptions occur
- B) It doubles read speed by default
- C) It works only for JSON files
- D) It unlocks quantum storage mode

Show Answer



Comprehensions

Compact, expressive transformations

List Comprehensions

```
# Classic loop
squares = []
for x in range(10):
    squares.append(x ** 2)

# List comprehension
squares = [x ** 2 for x in range(10)]
even_squares = [x ** 2 for x in range(10) if x % 2 == 0]
```

When to use

- Simple transform/filter in one expression
- Readability still remains high
- Avoid deep nesting inside one line



Dictionary Comprehensions

```
# Number → square mapping
square_dict = {x: x ** 2 for x in range(5)}

fruits = {"apple": 1, "banana": 2, "cherry": 3}
inverted_fruits = {value: key for key, value in fruits.items()}

inventory = {"apples": 30, "bananas": 50, "oranges": 10}
low_stock = {item: count for item, count in inventory.items() if count < 20}
```

Great for remapping and filtering dictionary data in one pass.



Generator Expressions and Memory

```
large_list = [x ** 2 for x in range(1_000_000)]  
large_generator = (x ** 2 for x in range(1_000_000))  
  
total = sum(large_generator)
```

Generators produce values lazily, one at a time.

Comprehension vs generator

List comprehension

Stores full result in memory.

Generator expression

Streams values on demand.

Use generators for large pipelines where results are consumed once.



Gotcha: List Multiplication Aliasing

✂ The trap

```
grid = [[0, 0, 0]] * 3
print(grid) # [[0, 0, 0], [0, 0, 0], [0, 0, 0]]
grid[0][0] = 9
print(grid)
# [[9, 0, 0], [9, 0, 0], [9, 0, 0]] # all rows changed
```

✓ The fix

```
grid = [[0, 0, 0] for _ in range(3)]
grid[0][0] = 9
print(grid)
# [[9, 0, 0], [0, 0, 0], [0, 0, 0]] # only row 0 changed
```

Why it happens

`[[0, 0, 0]] * 3` copies the **reference** three times, so every slot points to the same inner list. A comprehension creates a **new list object** on each iteration.

Rule of thumb: avoid using `*` to duplicate mutable nested objects. Use a comprehension instead.



Checkpoint: Comprehensions

Q5

What does this expression produce: `[x * 2 for x in range(5) if x % 2 == 0]`?

- A) `[0, 2, 4, 6, 8]`
- B) `[0, 4, 8]`
- C) `[2, 6]`
- D) It prints only on leap years

Show Answer

Q6

Which statement about generator expressions is true?

- A) They always create a list first
- B) They generate values lazily on demand
- C) They cannot be used with `sum()`
- D) They require a GPU

Show Answer



Live Coding: To-Do App

Iteration 8 and 9



Iteration Roadmap

Iteration 8

Handle non-numeric task IDs and avoid crashes.

Iteration 9.1

Persist tasks to `tasks.txt`.

Iteration 9.2

Persist structured tasks to `tasks.json`.

Goal: upgrade from a fragile in-memory script to a durable CLI mini-application.



Iteration 8: Robust Input Handling

```
def ask_task_id():  
    while True:  
        raw = input("Enter task ID: ")  
        try:  
            return int(raw)  
        except ValueError:  
            print("Please enter a numeric ID.")
```

User-facing errors should be clear and actionable, not raw tracebacks.



Iteration 9.1: Persist to tasks.txt

```
def load_tasks_txt(path="tasks.txt"):
    try:
        with open(path, "r") as f:
            return [line.strip() for line in f if line.strip()]
    except FileNotFoundError:
        return []

def save_tasks_txt(tasks, path="tasks.txt"):
    with open(path, "w") as f:
        for task in tasks:
            f.write(task + "\n")
```

Simple but limited for multi-field task structures.



Iteration 9.2: Persist to tasks.json

```
import json

def load_tasks_json(path="tasks.json"):
    try:
        with open(path, "r") as f:
            return json.load(f)
    except FileNotFoundError:
        return {}

def save_tasks_json(tasks, path="tasks.json"):
    with open(path, "w") as f:
        json.dump(tasks, f, indent=2)
```

Text vs JSON

- **TXT** quick for plain lines
- **JSON** better for nested metadata
- JSON reduces manual parsing errors

For a real to-do app with IDs, status, and descriptions, JSON is the safer long-term choice.



Summary

Error Handling

- Classify failure types
- Use specific exceptions
- Prefer graceful recovery

File I/O and JSON

- Choose modes intentionally
- Use `with open( ... )`
- Persist structured data with JSON

Comprehensions

- Express simple transforms compactly
- Use generators for large streams
- Protect readability over cleverness

Resilient programs are not error-free; they are error-aware and user-respectful.



Thank You for Your Attention!

"Errors should never pass silently. Unless explicitly silenced."

-- The Zen of Python

exceptions

files

json

comprehensions